power of graph theory. Graph theory provides us with many tools to measure distances and connectivity in graphs, many of which will be explored in the upcoming articles in this series.

In a graph, the edges may not always have weights. Moreover, it is possible to assign labels to edges, similar to the labels given to vertices. Now, let us create such a graph (for ease of reference, we call it *Graph_2*) using SageMath. Take a look at the SageMath code saved as *graph2.sagews*, provided below (line numbers have been included for clarity in the explanation).

```
1.  from sage.graphs.graph import
Graph

2.  edge_labels = {("A", "B"): "a",
("B", "C"): "b", ("C", "D"): "c",
("D", "E"):

"d", ("E", "A"): "e", ("E", "C"): "f"}

3.  adj_list = {
        "A": ["B"],
        "B": ["C"],
        "C": ["D", "E"],
        "D": ["E"],
        "E": ["A", "C"],
    }

4.  sgraph = Graph(adj_list)

5.  for edge, label in edge_labels.
items( ):
        sgraph.set_edge_label(edge[0],
edge[1], label)

6.  sgraph.show(edge_labels=True)
```

Now, let us go through the code line by line to understand its workings. In Line 1, similar to the previous program, the *Graph* class from the *sage.graphs.graph* module is imported. Line 2 creates a dictionary called *edge_labels* to store labels for the edges in the graph. Each key-value pair in this line represents an edge and its corresponding label. For example, *("A", "B"): "a"* means that the edge connecting vertices *A* and *B* will have the label *"a"*. Line 3 creates an adjacency list representation of the graph. It is a dictionary where each key is a vertex, and its value is a list of vertices that are connected to it. For example, *"A": ["B"]* means that vertex A is connected to vertex B. Notice that edges are not given weights in this example. Line 4 creates a new graph object called *sgraph* using the adjacency list defined in *adj_list*. This constructs the actual graph structure in SageMath. Line 5 has a loop that iterates through the dictionary *edge_labels* to set the labels for each edge in the graph. The body of the loop uses the *set_edge_label( )* method of the *sgraph* object to assign the label to the corresponding edge. Finally, Line 6 displays a visual representation of the graph *sgraph*. The parameter *edge_labels=True* ensures that the labels defined in *edge_labels* are displayed along with the edges while the graph is visualised. Use CoCalc to execute this code, and upon execution, the graph shown in Figure 4, called *Graph_2*, is obtained.

As mentioned earlier, a graph can also be represented using an incidence matrix. Now, let us examine the incidence matrix representation of the graph *Graph_2*. Add the following line of code at the end of *graph2.sagews* and execute it again.

```
print(sgraph.incidence_matrix( ))
```

On execution, the following additional output will be printed on the screen. But what does it mean?

```
[1 1 1 0 0 0]
[1 0 0 1 0 0]
[0 1 0 1 1 0]
[0 0 1 0 0 1]
[0 0 0 0 1 1]
```

Figure 5 will help us understand the working of an incidence matrix. An incidence matrix is a matrix where each row represents a vertex and each column represents an edge. For *Graph_2*, the vertices are A to E and the edges are from a to f (hence, the 5 X 6 matrix). If a vertex is connected to an edge, then the cell corresponding to that row and column of the incidence matrix will have a 1 as its value. Otherwise, the cell will have a 0 as its value. Since every edge has exactly two endpoints, every column of the incidence matrix will have exactly two 1's. In the next article in this series, we will do the reverse of what we did in this article. We will construct graphs by providing their adjacency matrix or incidence matrix.

Now, it is time to conclude our discussion. Graph theory, being almost three centuries old, has a lot of tools to offer. Hence, we will continue our exploration of the applications of graphs in the next article in this series as well. Although our discussion in this article focused on undirected simple graphs, there are other types of graphs, including directed graphs, multigraphs, mixed graphs, mixed multigraphs, and more. We will delve into these types of graphs also in the next article of this series. END

	a	b	c	d	e	f
A	1	0	0	0	1	0
B	1	1	0	0	0	0
C	0	1	1	0	0	1
D	0	0	1	1	0	0
E	0	0	0	1	1	1

Figure 5: The incidence matrix of *Graph_2*

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He maintains a technical blog.